

Milestone 2: Hazard Detection and Resolution

Forwarding is done in `gen_forward` and the load-use hazard is detected in `detect_hazard`, both in `pipeline.c`. The forwarding unit does not modify the operands directly: it sets mux select wires that the EX stage applies. The stall bubble and the pipeline flush are applied at the end of each cycle in `cycle_pipeline` in `pipeline.c`.

EX Hazard

An EX hazard occurs when the instruction entering EX needs a register that the previous instruction, now in the MEM stage, is going to write. The result already exists in the EX/MEM pipeline register. The condition in `gen_forward` is checked once per source register:

```
if (exmem->reg_write && exmem->rd != 0) {
    if (idex->use_rs1 && exmem->rd == idex->rs1) {
        pwires_p->forwardA = 1;
        fwd_exex_counter++;
    }
    if (idex->use_rs2 && exmem->rd == idex->rs2) {
        pwires_p->forwardB = 1;
        fwd_exex_counter++;
    }
}
```

This reads: the instruction in MEM writes a destination register that is not x0, the instruction entering EX actually uses this source register, and that destination equals our source. The `rd != 0` check plays the role of an x0 guard, since x0 is hard-wired to zero and an instruction writing x0 has no real destination. To resolve the hazard the forward select wire is set to 1 and the value to forward is latched (`pwires_p->exmem_fwd_val = exmem->alu_result`). The EX stage applies the mux before computing:

```
uint32_t val_rs1 = idex_reg.rs1_val;
if (pwires_p->forwardA == 1) val_rs1 = pwires_p->exmem_fwd_val;
else if (pwires_p->forwardA == 2) val_rs1 = pwires_p->memwb_fwd_val;
// same mux shape for val_rs2 with forwardB
```

The EX stage then computes with the forwarded value, so a dependent instruction directly after its producer runs without stalling.

MEM Hazard

A MEM hazard occurs when the needed register is written by the instruction two ahead, now in the WB stage. Its result sits in the MEM/WB pipeline register. The condition is:

```
if (memwb->reg_write && memwb->rd != 0) {
    if (idex->use_rs1 && memwb->rd == idex->rs1 &&
        !(exmem->reg_write && exmem->rd != 0 && exmem->rd == idex->rs1)) {
        pwires_p->forwardA = 2;
        fwd_exmem_counter++;
    }
    // same shape for rs2 -> forwardB = 2
}
```

It is the same match check, but against the MEM/WB register and with one extra term: the negated EX-hazard condition. If both older instructions write the same register, the EX hazard must win because the instruction in MEM is more recent, so the MEM hazard only fires for an operand the EX hazard did not cover. To resolve it the select wire is set to 2 and the value WB is writing back this cycle is latched.

For a load this is the data read from memory, otherwise the ALU result:

```
pwires_p->memwb_fwd_val = memwb->mem_to_reg ? memwb->mem_result
                        : memwb->alu_result;
```

The EX stage picks this value up through the same mux shown above when the select wire is 2.

Load-Use Hazard

Forwarding cannot help when a load is immediately followed by an instruction that uses the loaded register: when that instruction wants to enter EX, the load is still in MEM and the data has not been read yet. `detect_hazard` compares the load in the ID/EX register against the instruction in the IF/ID register:

```
if (idex->mem_read && idex->rd != 0) {
    uint32_t op = ifid->instr.opcode;
    bool uses_rs1 = false, uses_rs2 = false;
    uint32_t rs1 = 0, rs2 = 0;

    if (op == 0x33 || op == 0x63 || op == 0x23) {
        uses_rs1 = true; uses_rs2 = true;
        rs1 = ifid->instr.rtype.rs1; rs2 = ifid->instr.rtype.rs2;
    } else if (op == 0x13 || op == 0x03 || op == 0x67) {
        uses_rs1 = true;
        rs1 = ifid->instr.itype.rs1;
    }

    if ((uses_rs1 && idex->rd == rs1) || (uses_rs2 && idex->rd == rs2)) {
        pwires_p->stall = true;
        pwires_p->pc_src0 = regfile_p->PC;
        stall_counter++;
    }
}
```

`mem_read` is only true for loads, so the condition means: a load is in EX, it writes a real register, and the instruction behind it in ID reads that register as `rs1` or `rs2`. The opcode switch extracts the source register numbers of the ID instruction along with flags saying whether each one is really used. The flags matter because the `rs1` and `rs2` bit positions overlap immediate bits in other instruction formats, and without them an immediate could look like a matching register and cause a false stall. On a hazard we raise the stall wire and rewrite `pc_src0` to the current PC, so fetch repeats the same instruction next cycle. The dependent instruction is delayed one cycle, after which the load is in WB and normal MEM-hazard forwarding delivers the loaded value.

Pipeline Flush

Branches and `jal` are resolved in the MEM stage. The EX stage computes `branch_taken` and `branch_target` and they travel with the instruction through the pipeline registers. At the end of every cycle `cycle_pipeline` checks the value about to enter MEM/WB:

```
if (pregs_p->memwb_preg.inp.branch_taken)
{
    pwires_p->pcsrc    = true;
    pwires_p->pc_src1 = pregs_p->memwb_preg.inp.branch_target;
    branch_counter++;
    ifid_f.instr.bits = 0x00000013;    // and idex_f, exmem_f
    ...
}
```

When the branch is taken, the three instructions behind it were fetched from the wrong path and must have no effect. Each of the IF/ID, ID/EX and EX/MEM registers is replaced with a NOP: the instruction bits become 0x00000013, which is `addi x0, x0, 0`, all control signals are cleared to zero, and only the instruction address is kept so the trace still shows where the killed slot came from. At the same time `pcsrc` and `pc_src1` redirect fetch to the branch target.

Stall and Bubble Insertion

When `detect_hazard` raises the stall wire, `cycle_pipeline` inserts the bubble before the pipeline registers are latched:

```
pregs_p->ifid_preg.inp = pregs_p->ifid_preg.out; // hold in id
pregs_p->idex_preg.inp.reg_write = false;       // squash the copy
pregs_p->idex_preg.inp.mem_read = false;
pregs_p->idex_preg.inp.mem_write = false;
pregs_p->idex_preg.inp.branch = false;
pregs_p->idex_preg.inp.jump = false;
```

Two things happen. First, the IF/ID register is reloaded with its own current value, so the dependent instruction stays in ID and decodes again next cycle. Since the PC was rewritten, fetch also repeats, so nothing is lost. Second, the copy of the instruction that was already decoded this cycle becomes the bubble: it keeps its instruction bits, but every control signal that could cause an effect is cleared, so as it moves through EX, MEM and WB it cannot write a register, touch memory, or resolve as a branch. The held copy follows one cycle behind and gets the correct value through the MEM-hazard forward.

Other Changes

We added `rs1`, `rs2`, `use_rs1` and `use_rs2` to `idex_reg_t` so the forwarding and hazard units can see the source registers without re-decoding the instruction. `branch_taken` and `branch_target` were added to the EX/MEM and MEM/WB registers so the branch decision can travel to MEM, and the stall flag, `forwardA/forwardB` selects and the two forwarded values were added to the pipeline wires.

Branches now resolve in MEM instead of right away in EX like milestone 1. `stage_execute` just computes the outcome, using the forwarded operands so a branch right behind its producer compares the right values, and the redirect and flush happen at the MEM/WB boundary.

The order in `cycle_pipeline` is `fetch`, `detect_hazard`, `decode`, `gen_forward`, `execute`, `mem`, `writeback`, then the stall and flush handling. `detect_hazard` has to run after `fetch` so it can rewrite the PC, and `gen_forward` has to run before `execute` so EX sees the forwarded values.

At reset the pipeline registers hold 0x00000000, which is not a valid opcode, so a small `print_instruction` wrapper prints a blank line for zero bits instead of crashing in `parse_instruction`.

The ALU now handles `mulh`, `div` and `rem`, which milestone 1 decoded as `sll`, `xor` and `or` because it ignored `funct7`. The divide cases guard against a zero divisor since a wrong-path instruction still goes through the ALU before it gets flushed.